

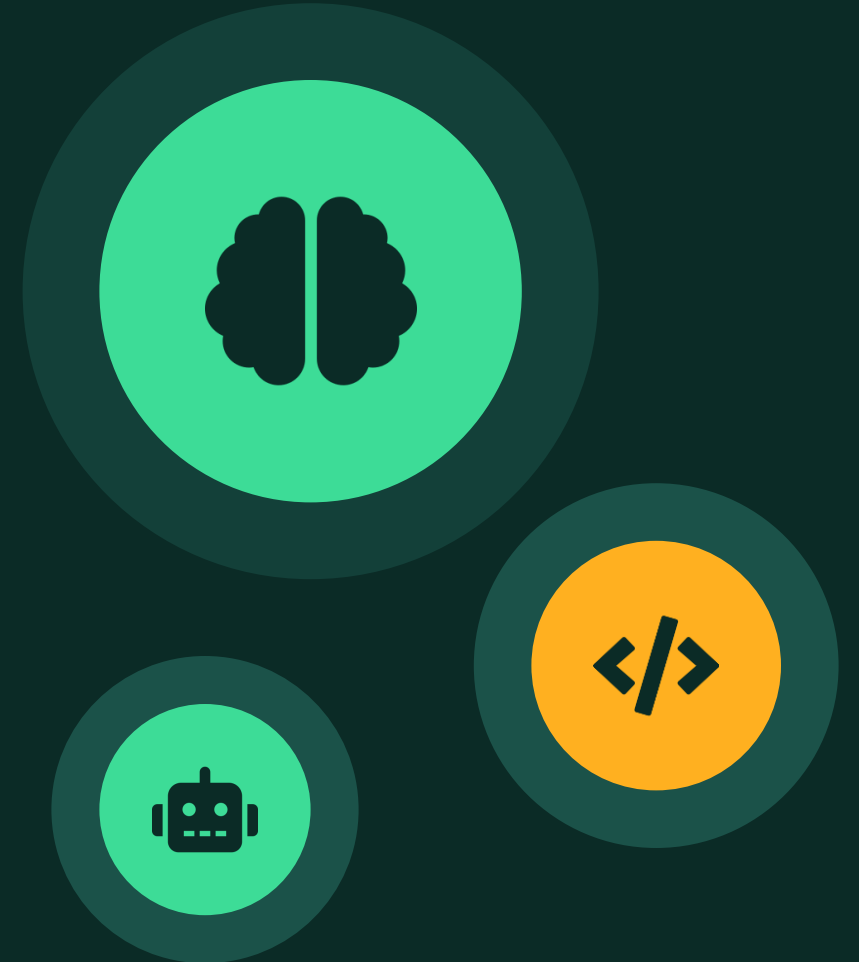
INTRA-COMPANY TRAINING · CGG SERVICES SAS

AI for Developers

Master AI to code faster

Day 1 of 2 · Developing effectively with AI

Trainer: Daouda DIOP



Your trainer



Daouda DIOP

Senior Data Scientist & Generative AI Expert

-  PhD in Physics – Optics & Computer Vision
-  R&D Project Manager – R&D Engineer, Optics/Computer Vision
-  Senior Data Analyst / Data Scientist
-  Certified Trainer – LLM, RAG, AI Agents

CAREER PATH · IN ORDER



What you will be able to do



Use AI to produce, fix, test and refactor code on real-world cases



Choose the most suitable AI tools based on the project, the team and security constraints



Structure a development workflow that integrates AI in a reliable and reproducible way



Industrialize AI usage with documentation, code review and development cycle automation



Assess the quality, limitations and risks of generated code before shipping to production

How we will work together

2 days

14 hours, live and remote

~50%

hands-on practice

1

common-thread project

Certificate

of success, sent by email

How you are assessed

- Hands-on work, shared with me in real time (screen sharing, GitHub...)
- End-of-training report by email, with my assessment
- Certificate of success

What we expect from you

- Regular hands-on development experience on a real project
- Comfortable with Git and with reading code
- A modern IDE or code editor, used daily
- Your laptop

Two days at a glance

DAY 1

Developing effectively with AI

- 1 Understanding how AI actually codes
- 2 Effective prompting for developers
- 3 Overview of AI tools for developers
- 4 A complete AI-driven development workflow

HANDS-ON



A complete feature with AI

DAY 2

Industrializing AI usage

- 1 Using AI agents for complex tasks
- 2 Integrating AI into your team workflow
- 3 Automating the development cycle with AI
- 4 Choosing your AI development stack

HANDS-ON



Industrializing the project with AI

Warm-up: where are you today?



Your tools

Which AI tools do you use?
Chat, editor assistant,
agent... or none yet.



Your usage

For which tasks, and how
often? Code, tests, docs,
debugging?



Your best experience

When did AI really save
you time?



Your worst experience

What went wrong, and
why?



What do you want to leave with after these two days?

DAY 1 • MODULE 1

Understanding how AI actually codes

From LLMs to agents: what really happens when AI writes your code.



What is an LLM?

An LLM is a neural network trained to **predict the next token.**



Large

Typically billions of parameters:
numbers adjusted during training.



Language

Trained on huge amounts of text,
including a lot of source code.



Model

A mathematical function: text goes
in, probabilities come out.



It is not a search engine and not a database of answers. It generates, it does not look up.

Predicting the next token, one step at a time

What the model has received

```
def add(a, b):  
    return a +
```



Next-token probabilities (illustrative)



Its job: score every possible next token

Then one token is chosen, appended, and the process repeats.



Plausible code is code that is likely, not code that has been checked.

Tokens: what an LLM actually reads and writes

Illustrative split of a function name into tokens



The exact split depends on each model's tokenizer.



A rough rule

In English, one token is roughly three quarters of a word. Code, rare words and other languages often need more.



Everything is counted in tokens

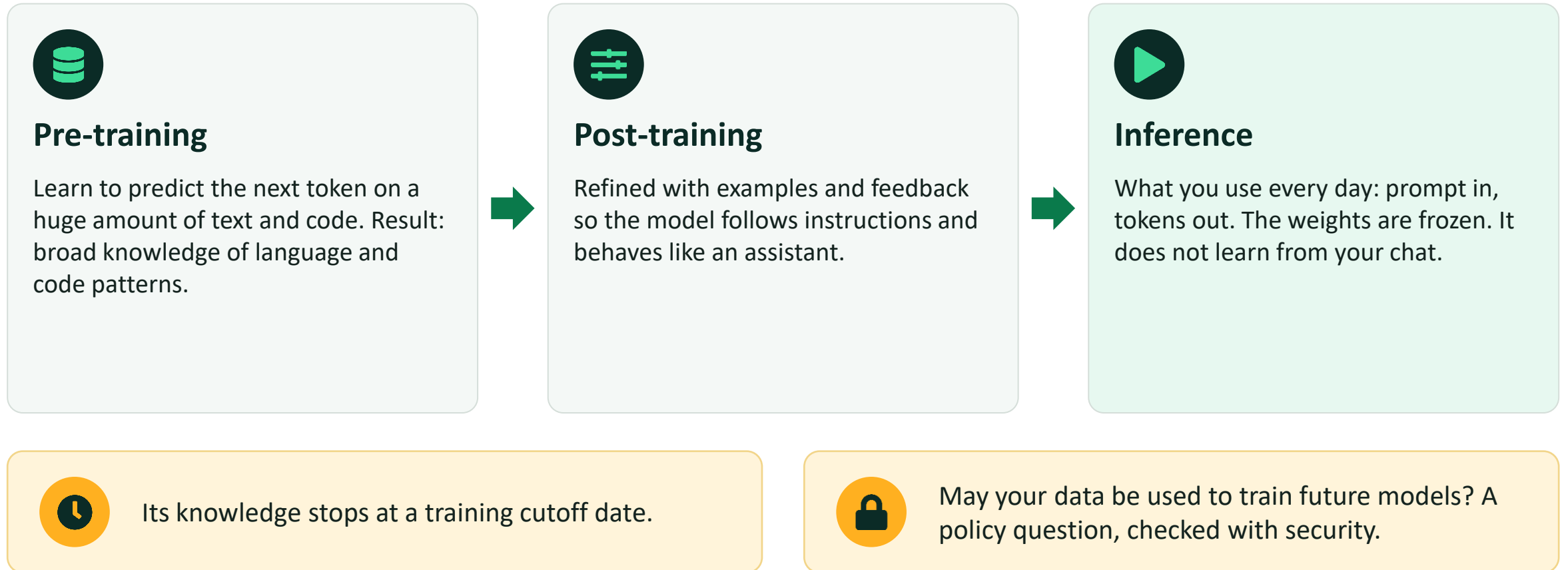
Context size, speed and, usually, price.



Not letters

The model sees tokens, not characters: counting letters can be unreliable.

How an LLM is built: three stages



How the model “reads” your context: attention

```
1  user = load_user(user_id)

2  orders = fetch_orders(user)

3  total = compute_total/orders)

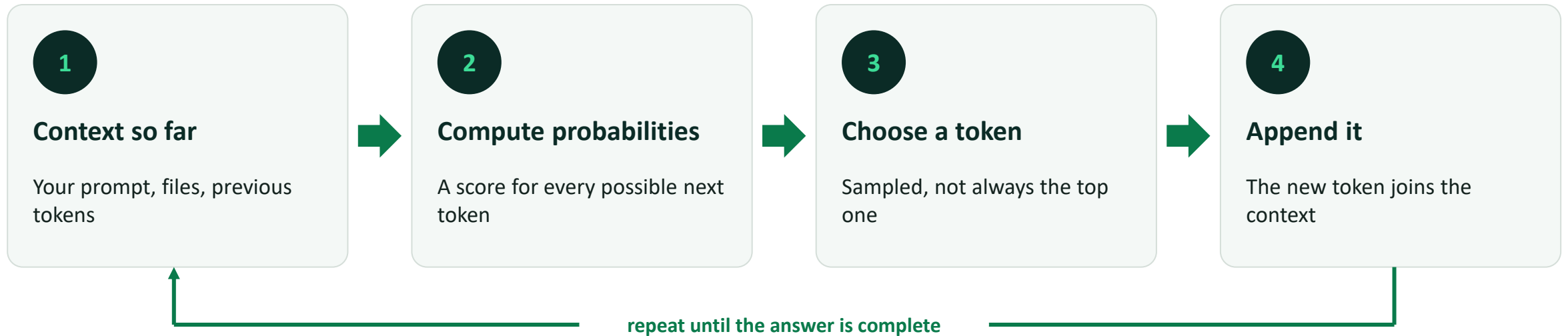
4  return user.
```

“user.” looks back at line 1

- **Transformer (2017).** Each new token can look back at all previous tokens in the context.
- **It learns what matters.** For the current token, which earlier tokens deserve attention?
- **Good fit for code.** A name defined at the top, used at the bottom.

The model can only attend to what is in its context. If it is not there, it does not exist for the model.

Generating code: a loop, with a little randomness



Temperature

Lower: more predictable. Higher: more varied. Not always exposed in your tool.



Same prompt, different code

Do not count on identical output twice.



Make the process reproducible

Same template, same tests, same review.

The context window: the model's working desk

Everything below must fit in the window: input and output together



One shared window

Input and output share the same space. Long answers need room.



When it is full

The tool drops or summarizes older content. Earlier instructions can be lost.



Relevant beats big

A long, noisy context tends to make the model less focused.

Memory: the model remembers nothing

Turn 1



Turn 2: everything is sent again



- **Stateless.** The application re-sends the conversation every time.
- **“Memory” features** are saved text, inserted back into the context.
- **New session = blank slate.** Unless you reload what matters.
- **Important knowledge lives in files:** docs, project rules, tests.



If it is not in the context, it does not exist for the model.

Six limits to keep in mind



Knowledge cutoff

May not know recent library versions or APIs unless you say so.



Blind to your code

It only sees what you or your tool put in the context.



Cannot run anything

A plain LLM cannot compile, test or execute. It writes text.



Non-deterministic

The same prompt can produce different outputs.



Hallucinations

Fluent, confident, sometimes false.



Finite context

Long tasks can overflow the window.

Hallucinations: confident, fluent... and wrong



AI assistant

Sure! Just use `requests.get_json(url)`. It downloads the page and returns the parsed JSON directly.

This function does not exist

Example written for this course, not from a real session.

Why does it happen?

- The model generates what is plausible, not what is verified.
- There is no built-in fact-checking step.
- A confident tone says nothing about correctness.
- Gaps in knowledge get filled with likely-looking guesses.



In code: invented functions, parameters, options and packages. Good news: code is checkable.

Eight common errors in AI-generated code

1

Invented APIs

Functions or packages that do not exist. Verify every dependency.

2

Outdated usage

Correct for an older version of the library.

3

Plausible wrong logic

Off-by-one errors, inverted conditions.

4

Missed edge cases

Null, empty, time zones, concurrency.

5

Insecure patterns

Weak validation, injection risks, secrets in code.

6

Ignores conventions

Style or architecture not matching your project.

7

Over-engineering

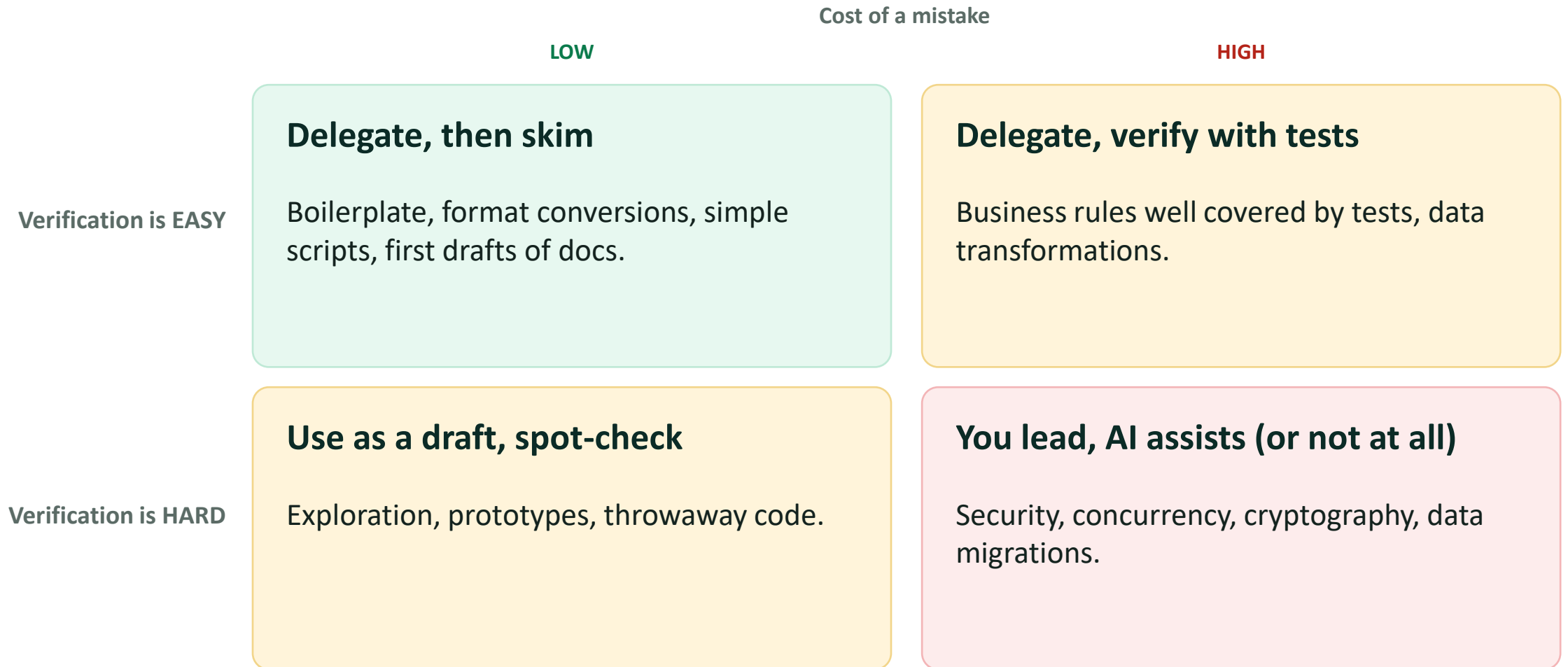
More layers and abstractions than needed.

8

Empty tests

Tests that pass but prove nothing.

Calibrating trust: how much should you check?



Five checks before accepting generated code

1

Do I understand every line?

2

Does it run, and do meaningful tests pass?

3

Do the dependencies and APIs really exist, in our version?

4

Does it match our conventions and architecture?

5

Is it safe: inputs, secrets, permissions, licenses?



**You own what you
commit.**

Whoever wrote the first draft.

When not to use AI



Confidential data

Never paste secrets, customer data or protected code into a tool that is not approved.



You cannot evaluate the result

No expertise and no tests: you cannot tell good from plausible.



A trivial change

Faster to do by hand than to explain.



Critical code, no expert review

Security or safety-critical logic needs a specialist.



Unclear requirements

Clarify with people first. AI will build the wrong thing, fast.



Learning is the goal

To master a skill, struggle a little first.

Quick check: true or false?

1

An LLM learns from your conversation while you chat.

2

A more confident answer is more likely to be correct.

3

If something is not in the context, the model cannot pay attention to it.

4

Asking the same question twice always gives the same code.

5

The context window includes both your input and the model's output.



**True or false? Answer
in the chat.**

DAY 1 • MODULE 1 • PART B

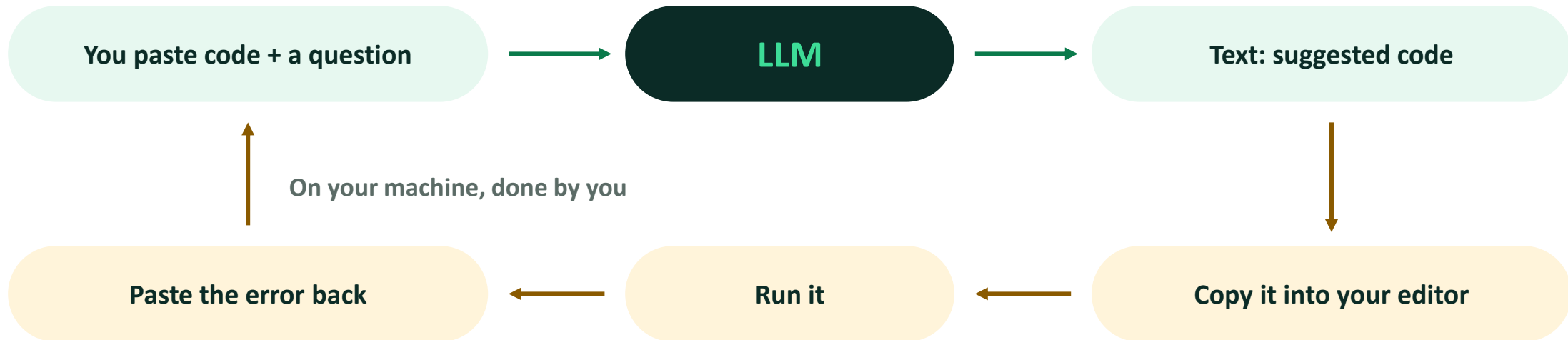
From LLM to Agent

What changes when a model gets tools and a loop.



An LLM alone: text in, text out

In the chat



In this setup, YOU are the loop. The model sees no files, runs nothing, and never sees the result.

Giving the model hands: tools

1

The model is told which tools exist

Names, descriptions, parameters.

2

It replies with a structured request

“Call this tool with these arguments.”

3

The harness executes it

Then puts the result back into the context.

read_file

search_code

edit_file

run_command

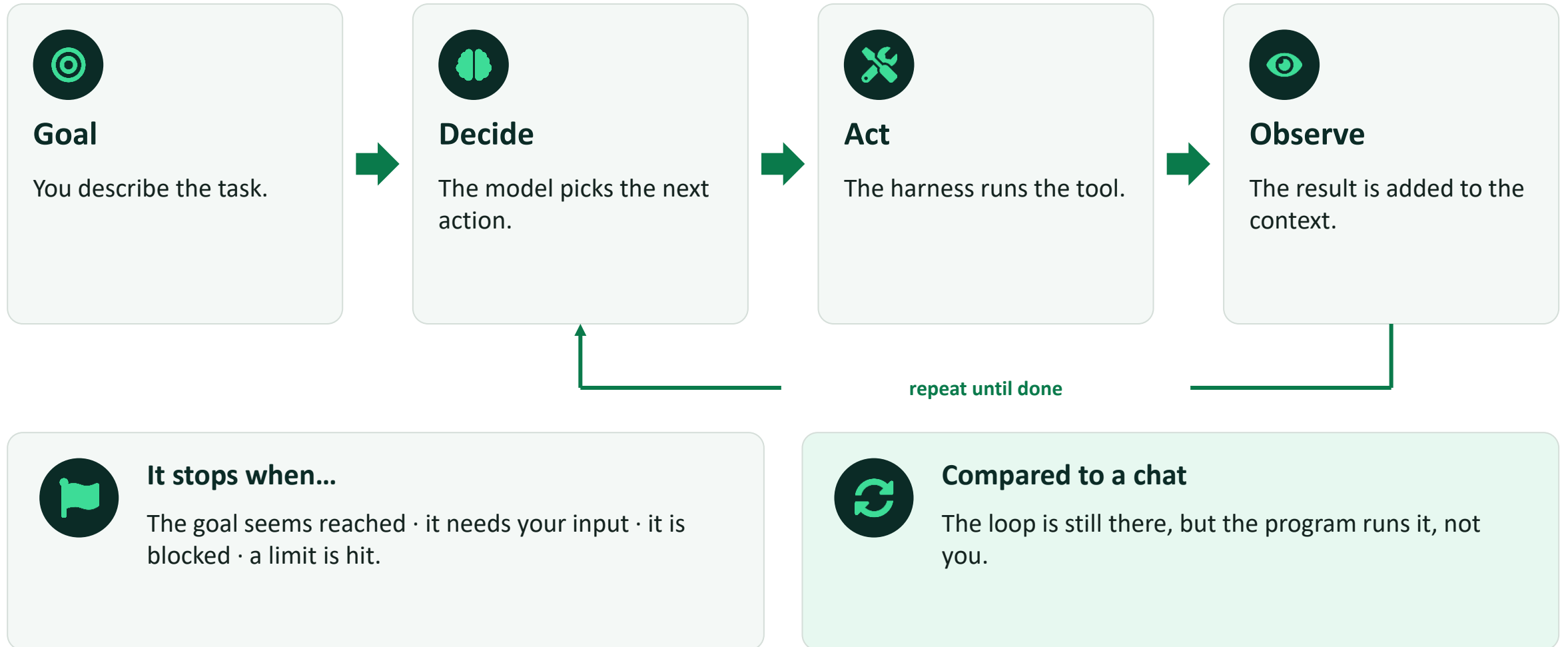
MODEL OUTPUT

```
// illustrative format
{
  "tool": "read_file",
  "arguments": {
    "path": "src/orders.py"
  }
}
```



The model never executes anything. It only asks.

The agent loop



LLM vs agent: what really changes

	LLM in a chat	Agent
What it is	A model that turns text into text	A model + tools + a loop
Who drives	You: copy, paste, run, report back	The agent acts and retries; you supervise
Access to your code	Only what you paste	Searches and reads files itself
Can it act?	No, it only writes text	Yes: edits files, runs commands (with permissions)
Feedback	You relay the errors	Reads test output and errors directly
Best for	Questions, design talks, snippets, explanations	Multi-step, multi-file tasks
Main risk	A wrong answer you may not notice	Wrong actions at scale, unwanted changes, cost

Anatomy of an agent

Agent = Model + Tools + Loop + Context + Guardrails



Model

Reads the context and decides the next step.



Tools

Read and search files, edit files, run commands and tests.



Loop (harness)

The program that calls the model, runs tools, feeds results back.



Context

System instructions, project rules, files read, results so far.



Guardrails

Permissions, sandbox, approvals: what it may do alone.



The harness matters a lot: two tools built on the same model can behave differently.

One real request, step by step

YOU

Ask: “Fix the failing test test_parse_date.”

HARNESS

Builds the context: system instructions + project rules + tool list + your message. Sends it to the model.

MODEL

Replies with a tool request: run_tests.

HARNESS

Runs the tests, adds the failure output to the context, sends everything to the model again.

MODEL

Requests read_file on the function, then edit_file with a patch. The harness executes each one.

HARNESS

Runs the tests again: they pass. The model writes a short summary.

YOU

Review the diff. Accept, adjust or reject.

Illustrative sequence. The exact steps vary by tool and by task.

Three things to remember about this loop

1

The model only produces text

The harness does everything else:
reading, editing, running.
Permissions live in the harness.

2

Each step re-sends a growing context

Cost, latency and noise grow with every step. Keep tasks small.

3

An agent is only as good as its feedback

Tests, compiler errors and linters let it correct itself. Without them, it guesses.



This is why we keep agent tasks small, and always give the agent a way to check its own work.

A spectrum of autonomy



Chat

You do the steps.

The AI advises.



AI-assisted IDE

The AI proposes or applies edits in your editor.

You approve each change.



Autonomous agent

The AI plans and executes multi-step tasks.

You review the result.

More manual control

More delegation, more supervision needed

Many tools cover several points on this spectrum.

Exercise: LLM or agent?

- A** You paste a regex into a chat window and ask what it does.
- B** You ask a tool to find where invoices are validated, add a rule, and run the test suite.
- C** Code suggestions appear as you type in your editor.
- D** You paste a stack trace and ask for possible causes.
- E** A tool reads a failing CI log, edits the code, re-runs the tests and proposes a fix.

Test: does something other than the model take actions, observe the results and loop?

DAY 1 • MODULE 2

Effective prompting for developers

From vague requests to reliable, reusable prompts.



A prompt is a specification



Think of a very fast colleague...

...who has read a lot, but has never seen your project and remembers nothing from yesterday.

THE TEST

If a human colleague could not do the task from your message alone, the model cannot either.

Be specific

Give the context it cannot guess

Define what “done” looks like

Why prompts fail, and how to fix them

Symptom	Usual cause	Fix
Generic, useless answer	Vague goal	State the outcome and the reason
Code that does not fit the project	Missing context	Give stack, versions, relevant files, an example to follow
Half-done or messy result	Too many tasks at once	One task per prompt; split big ones
Endless back-and-forth	No definition of done	Add acceptance criteria and tests
Unwanted changes everywhere	No constraints	Set the scope: what to touch, what not to touch

The anatomy of a good prompt: six blocks

1

GOAL

What do I want, and why?

One clear task. The outcome, not just the action.

2

CONTEXT

What does it need to know?

Stack and versions, relevant code, errors, existing patterns.

3

CONSTRAINTS

What must or must not happen?

Scope, style, allowed libraries, files not to touch.

4

EXAMPLES

What does good look like?

Sample input/output, or a function to imitate.

5

FORMAT

How should the answer be shaped?

Diff only? Steps then code? Length, language.

6

VERIFICATION

How will we know it is right?

Acceptance criteria, tests, assumptions, open questions.

Optional: a role (“act as a senior reviewer”) can set the tone, but it does not replace context.

From weak to strong: the same request, rewritten

WEAK PROMPT

“Fix my code, it doesn’t work.”

What the model must guess:

- Language and version
- The error message
- Expected behavior
- What may be changed
- What “fixed” means

STRONG PROMPT

GOAL

Fix the failing test `test_parse_date_iso` (`tests/test_dates.py`).

CONTEXT

Python 3.11, pytest. Function: `parse_date` in `utils/dates.py`.

Error: `ValueError: time data`

`'2024-03-01T00:00:00Z'` does not match format `'%Y-%m-%d'`

Expected: `date(2024, 3, 1)`

CONSTRAINTS

Smallest possible change. Standard library only.
Do not modify the test.

FORMAT

Root cause in 2-3 sentences, then a unified diff.

VERIFICATION

Give the command to run the tests and list possible side effects.

Context: give the model the right facts



Include

- Stack, language, versions
- The relevant code: function, signature, types
- The exact error or stack trace, copied
- Existing patterns to follow
- Business rules that matter here



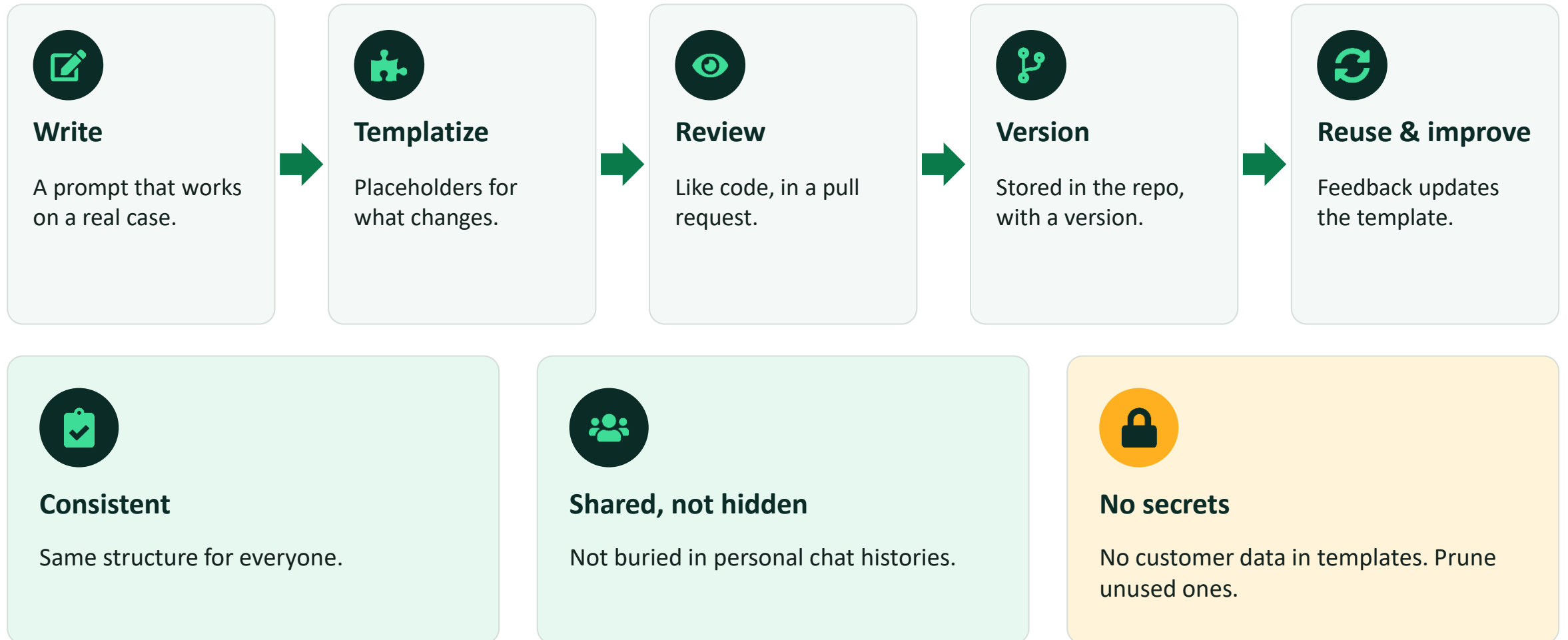
Leave out

- Whole-repository dumps
- Unrelated files and old failed attempts
- Secrets, credentials, personal or customer data
- Guesses presented as facts



Relevant beats big. Most tools can reference files directly: use that instead of pasting everything.

From a personal prompt to a team asset



Example: a team prompt template

prompts/bugfix.md

```
# Bugfix prompt - v1.2
## Goal
Fix: {{bug_summary}}
## Context
- Stack: {{language_version}}, {{framework}}
- Relevant files: {{files}}
- Error or stack trace: {{trace}}
- Expected vs actual: {{expected}} / {{actual}}
## Constraints
- Smallest possible change; no unrelated files
- Do not modify existing tests unless stated
- Follow the conventions in {{conventions_file}}
## Output
1. Root cause (max 3 sentences)
2. Diff    3. Test added or updated
## Verification
- Command to run the tests
- Risks and possible side effects
```



Placeholders

{{...}} make it reusable.



Team rules, once

Constraints encode conventions for everyone.



Reviewable answers

The Output block shapes what comes back.



Versioned

The version in the title shows it evolves.

Don't repeat yourself: project instruction files

- **Stable information goes in a file in the repo**, loaded automatically by the tool.
- **Names vary by tool**: CLAUDE.md, AGENTS.md, or an editor rules file.
- **Put in it**: build, test and lint commands; architecture in a few lines; conventions and “never” rules; links to docs.
- **Keep it short and current**. Review it like code.

EXAMPLE ONLY

```
# Project guide
## Commands
- Run tests: pytest -q
- Lint: ruff check .
## Architecture
- api/          HTTP layer
- services/     business logic
- db/           data access
## Conventions
- Type hints everywhere; small functions
- api/ never calls db/ directly
## Rules
- Add or update tests with every change
- Never commit secrets
```

Reliable code generation: six tactics



1 • Plan first

Review the approach before any code exists.



2 • Small steps

One function or file at a time. Verify each step.



3 • Show an example

Point to an existing function to imitate.



4 • Test first

Give the expected behavior as a test, then generate.



5 • Assumptions and questions

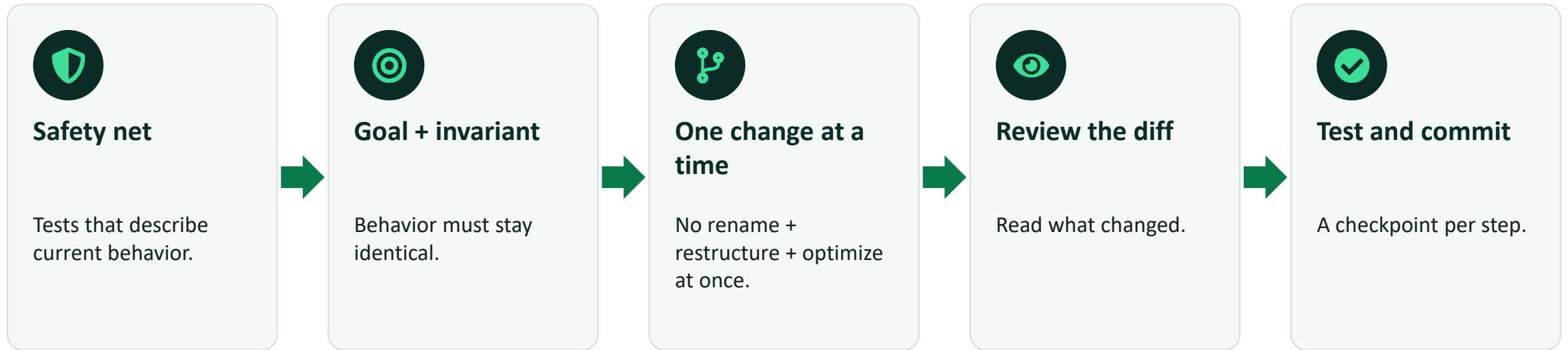
“List your assumptions and ask me if anything is unclear.”



6 • Types and signatures

Interfaces reduce guessing.

Guided refactoring



EXAMPLE PROMPT

```
Refactor process_order() in services/orders.py.  
Goal: split it into smaller functions and remove duplication.  
Invariant: behavior must stay identical; all existing tests pass.  
Constraints: no new dependencies; do not change public signatures.  
Work in small steps and show me the diff for each step.
```

AI-assisted debugging: evidence, then method

Give it the evidence

- Symptom, error message, stack trace
- Expected vs actual behavior
- A minimal reproduction
- Recent changes
- What you already tried

1

Ask for ranked hypotheses, each with a way to test it

2

Run the checks (you, or the agent) and feed back the results

3

Fix only once the root cause is clear

4

Add a regression test



Ask for the cause before the fix. A hypothesis is a lead, not a fact: confirm it with a check.

Writing tests with AI

1

Ask for a test plan first

Behaviors, edge cases, error paths.

2

Generate tests from the plan

3

Read the assertions

Do they check something real?

4

Run them, then break the code on purpose

They must fail. If not, they prove nothing.

5

Remove redundant or brittle tests

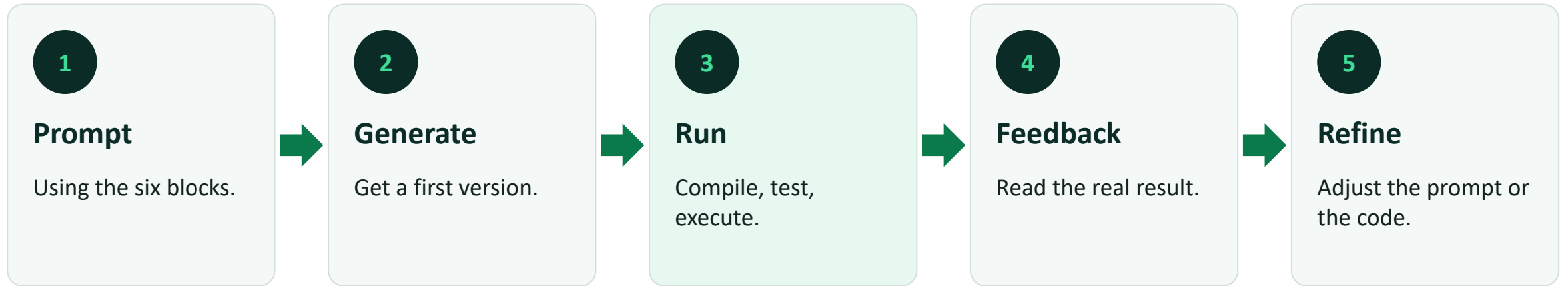


A green test is not necessarily a good test.

Watch for:

- Tests that copy the implementation logic
- Assertions that can never fail
- Excessive mocking
- Tests adapted to make wrong code pass

Efficient iteration loops



Paste exact errors

The precise message, not a description of it.



One change per turn

Commit checkpoints so you can go back.



Know when to restart

After 2–3 failed attempts, start fresh with a better prompt. A long chain of failures pollutes the context.

Exercise: rewrite a weak prompt

WEAK

“Make this faster.”

WEAK

“Write tests.”

WEAK

“It doesn’t work, fix it.”

1

Pick one, or bring a real prompt of your own

2

Rewrite it with the six blocks

3

Run both versions in your AI tool

4

Compare, and share what changed

Reminder: no secrets and no confidential data in your prompts.

DAY 1 • MODULE 3

Overview of AI tools for developers

Chat, IDE assistants and autonomous agents, and how to choose.



Three families of AI tools

	Chat	AI-assisted IDE	Autonomous agent
Where it lives	Browser or app conversation	Inside your editor	Works on your repo, often from the terminal
Examples	General-purpose chat assistants	Cursor, Copilot...	Claude Code, Codex, Opencode...
Context comes from	What you paste or upload	Open files and workspace (varies)	Explores the repo itself
Strengths	Exploring, learning, explaining, design talks	Fast in-flow edits and completions; easy to review	Multi-file tasks; runs tests and iterates
Limits	Manual context; you copy results back	Long multi-step work is harder to steer	Needs supervision and guardrails; time and cost

Typical characteriztics, not guarantees. Tools evolve quickly, and many IDE tools now include agent modes.

Criteria for choosing your tools



Task type and size

Question, in-flow edit or multi-file change?



Project and ecosystem

Language, framework, codebase size.



Confidentiality

Security first: where does your code go?



Team size and skills

What can the team use and maintain?



Integration

Editor, Git workflow, CI/CD.



Cost and licensing

Per user or per usage; how predictable?



Control and auditability

Can you see and control what it does?



Flexibility and lock-in

How hard is it to switch later?

Security questions to ask before adopting a tool



What data leaves the machine, and where is it processed?



Is it retained? For how long?



May it be used to train models?



Deployment options: cloud, private, local models?



Access control, single sign-on, audit logs?



What can the agent touch: files, shell, network, secrets?



Who owns the generated code? What about licenses?



**Your company's policy
comes first.**

Take these questions to your IT and security teams. Do not assume the answers.

Which tool for which job?

The job	A good starting point	Why
Quick question, explanation, brainstorm	Chat	Low friction, no setup
Writing and editing code in your flow	AI-assisted IDE	Suggestions where you work
Multi-file feature, refactoring, batch of tests	Autonomous agent	Explores, edits, runs and iterates
Sensitive code or data	Only tools approved by your company	Policy first
Not sure	Start small and read every diff	You build judgment by checking results

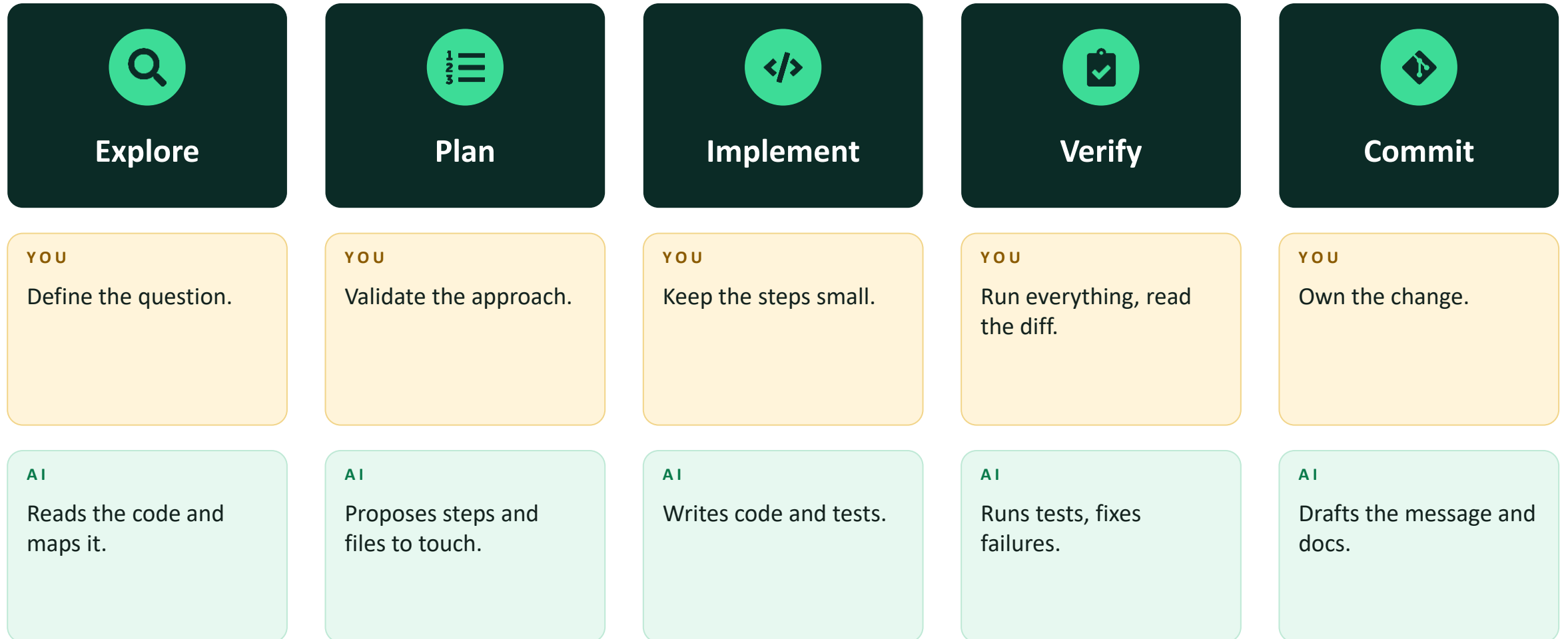
DAY 1 • MODULE 4

A complete AI-driven development workflow

Explore, plan, implement, verify, commit.



The loop: explore, plan, implement, verify, commit



Understanding an existing codebase

PROMPT IDEAS

"Give me a map of this project: entry points, main modules and how data flows."

"Where do we validate invoices?
Cite files and line numbers."

"Explain this function and list its callers."

"Which tests cover it?"



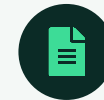
Ask for evidence

File paths and line numbers.



Open them and check

A convincing summary can be wrong.



Write findings down

A short doc in the repo for the team.



Low risk, high value: reading code is the safest place to start with AI.

Implementing a full feature

1

Write a mini spec

Goal and acceptance criteria.

2

Ask for a plan, review and correct it

Files, steps, risks. Before any code.

3

Implement step by step

Not everything at once.

4

Add or run tests with each step

5

Review the diff, commit per step

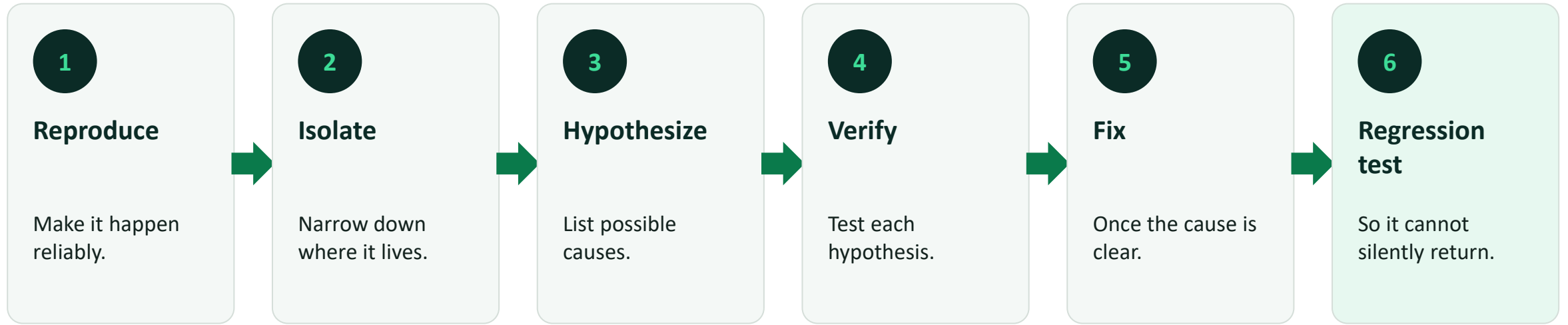
Small, reviewable commits.



The plan is where you have the most leverage.

A wrong plan produces wrong code, very efficiently.

Fixing a complex bug



Where AI helps

Reading logs and traces, proposing hypotheses, suggesting where to add logging, writing the reproduction test.



A tip for agents

Ask for a reproduction test first. A failing test is the best bug report: a precise, executable definition of success.

Cheat sheet: one pattern per job

Job	Start with	The AI’s job	Your check
Understand code	A precise question	Map the code and cite files	Open the files it cites
New feature	Mini spec, then a plan	Implement in small steps	Tests and the diff
Bug	A failing reproduction	Hypotheses, then a fix	Regression test
Tests	A list of behaviors	Generate the cases	Break the code on purpose
Refactoring	Tests locking behavior	Small, one-kind steps	Run tests after each step

Hands-on: develop a complete feature with AI



Explore an unknown codebase with AI



Implement a multi-file feature



AI-assisted debugging and refactoring



Generate and improve tests



Compare an AI-assisted IDE with an autonomous agent

RULES OF THE GAME

- Work on a branch
- Commit often
- Keep notes: what worked, what did not
- Be ready to share your screen
- No secrets in prompts



Debrief: what did you observe?



Where did AI save you time?

Which tasks, and why?



Where did it mislead you?

Weak prompt, missing context, model limit?



IDE assistant or agent?

When did you prefer each one?



What should we share?

Prompts or rules for a team template or instruction file.

Day 1 in five sentences

- 1 An LLM predicts tokens. Plausible is not the same as true.
- 2 The context is everything the model knows about your task. Relevant beats big.
- 3 An agent = model + tools + loop. The harness acts, the model asks.
- 4 A good prompt: goal, context, constraints, examples, format, verification.
- 5 Verify with tests and diff review. You own what you commit.

Tomorrow: from personal effectiveness to team industrialization.